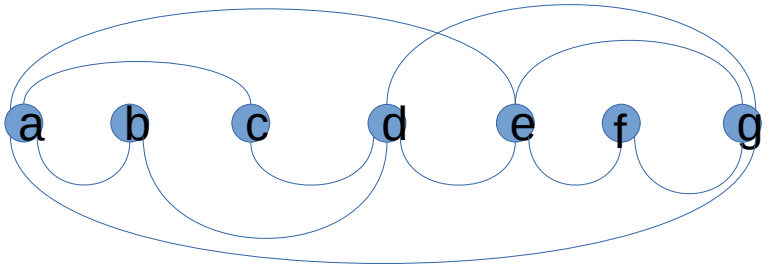
Algoritmo de Dijkstra

Caminho de custo mínimo

Em muitas situações é útil determinar o caminho mais curto entre duas localizações, quando se trata de arestas de mesmo peso ou sem ponderação a determinacão do caminho mínimo se resume à busca pelo caminho que percorra a menor quantidade de arestas entre a origem e o destino.

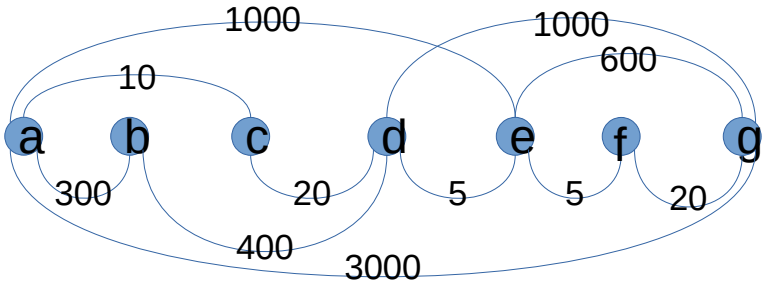
(caminho mínimo = menor caminho = caminho mais curto)



caminho mínimo a→g: custo de 1 aresta

Porém, quando as arestas do grafo são ponderadas, o caminho mínimo não necessariamente será o mais curto em termos da quantidae de arestas. Nesse caso a determinação do caminho mínimo precisa levar em conta o "custo" total desses pesos na determinação do caminho mínimo.

(caminho mínimo = caminho de menor peso total)



caminho mínimo a→g: custo = 60 (5 arestas)

Caminho de custo mínimo

Uma solução geral para o problema do caminho mínimo é implementada pelo algoritmo de Dijkstra, o qual trata um grafo com arestas com pesos positivos e calcula o caminho mínimo entre dois vértices pré-determinados.

Dijkstra não é a única opção para a determinação do caminho mínimo, o algoritmo de Bellman-Ford é outro muito citado na literatura para este problema.

Caminho de custo mínimo

Vamos agora ao algoritmo de Dijkstra apresentado na seção CAMINHO MÍNIMO E ÁRVORE GERADORA MÍNIMA do livro da Judith Gersting (disponível no formato ebook e fisicamente no acervo da biblioteca do CCT).



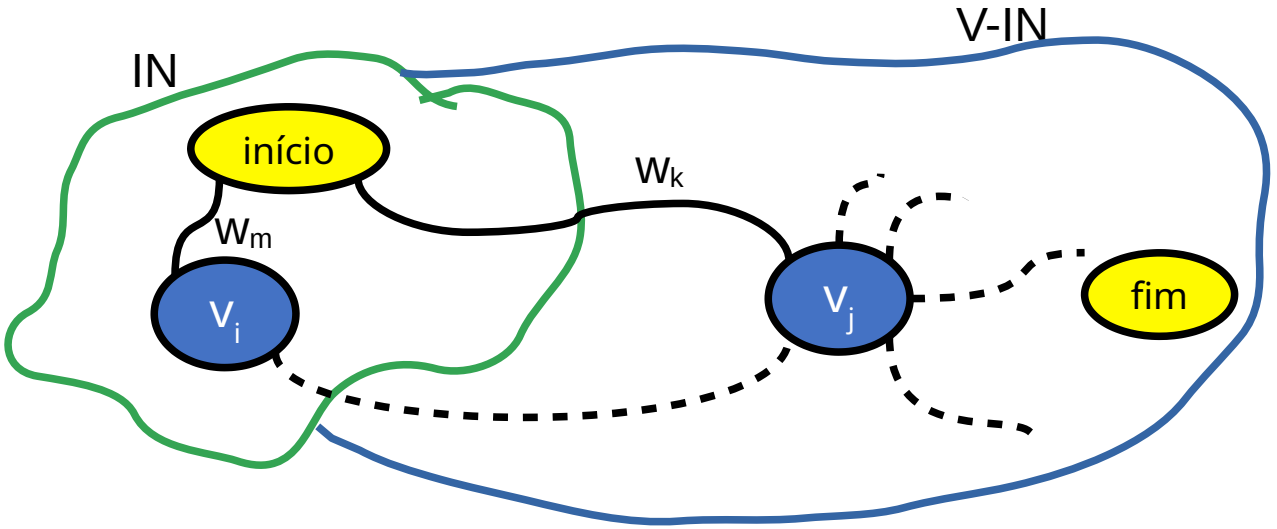
Fundamentos Matemáticos para a Ciência da Computação

Judith L. Gersting

Demonstração (fonte Wikipedia)

Dijkstra

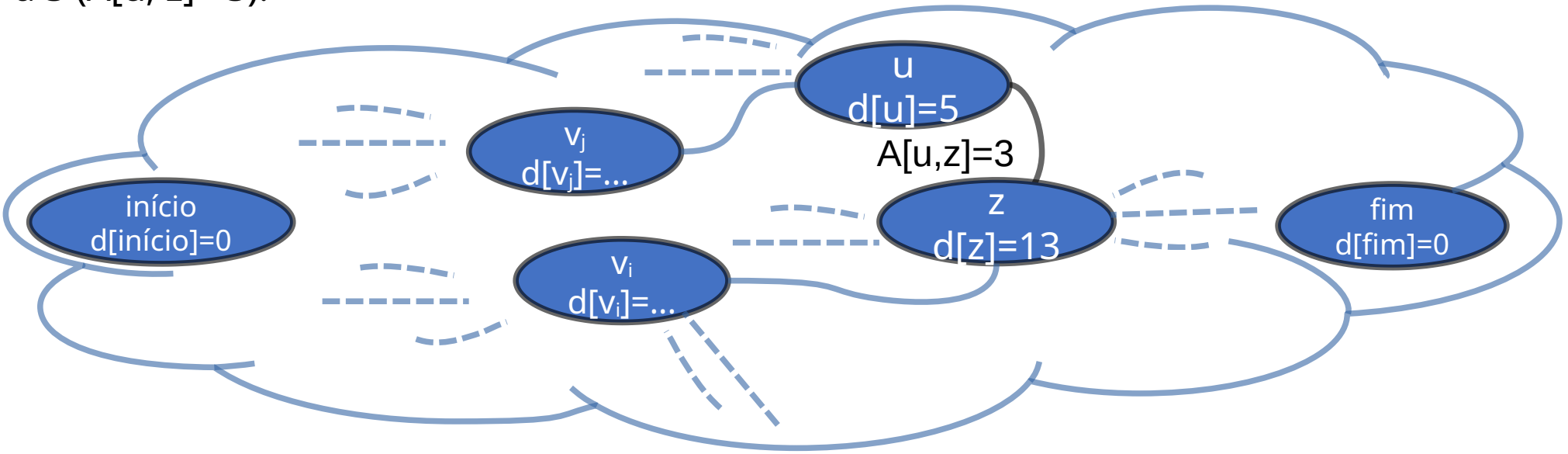
- Seja G(V,E) um grafo ponderado, Dijkstra mantém um conjunto IN de vértices já processados;
- O algoritmo seleciona repetidamente um vértice v_j ainda não processado ($v_j \in \{V - IN\}$) e avalia o custo para alcançar esse v_j a partir de um outro vértice ainda já processado;
- Essa avaliação analisa as alternativas para alcançar v_j mantendo o objetivo de construção de uma parte de um possível caminho mínimo que alcance este vértice (v_j);
- Este processo de avaliação das alternativas de caminho em termos do custo total é chamado de relaxação.



Dijkstra

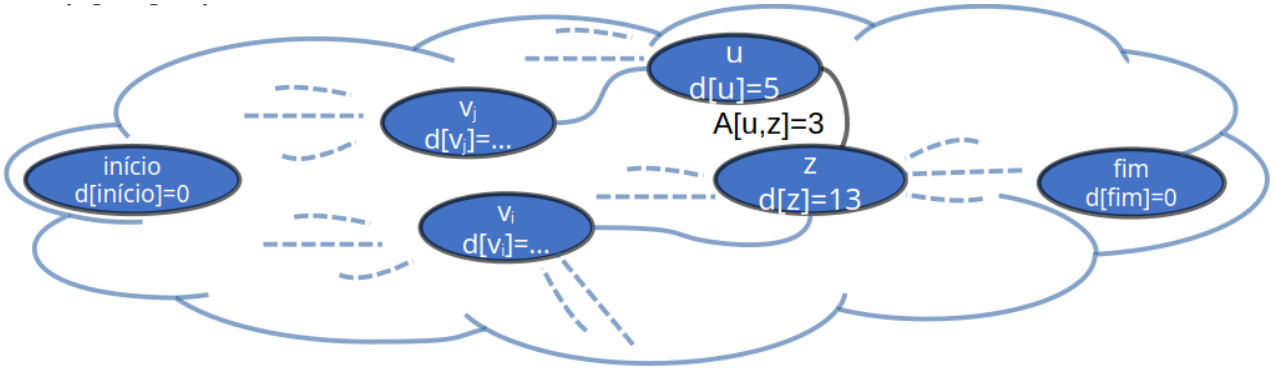
Seja *A* a matriz de adjacências de um grafo G(V,E) ponderado.

Suponha que o custo mínimo atual para alcançar *u* seja d[*u*]=5, o custo mínimo atual para alcançar z a partir de v_i seja d[z]=13, bem como o peso da aresta de u a z seja igual a 3 (A[u, z]= 3).



	...	u	z	...
d Custo acumulado	…	5	13	…
s vértice do trecho	...	v_j	v_i	

Dijkstra



Deseja-se saber se o caminho até *z* via *u* leva a um custo menor do que caminho atual até *z via* v_i (este custo atual é d[z]=13), ou seja, será que existe um caminho mais curto até *z* via *u* ?

Para tanto:

if (d[z] > d[u]+A[u,z]) // d[z] atual é maior do que acessar Z por meio de u?

então: d[z]=d[u]+A[u,z] // sim, então a nova d[z] será aquela obtida via *u*

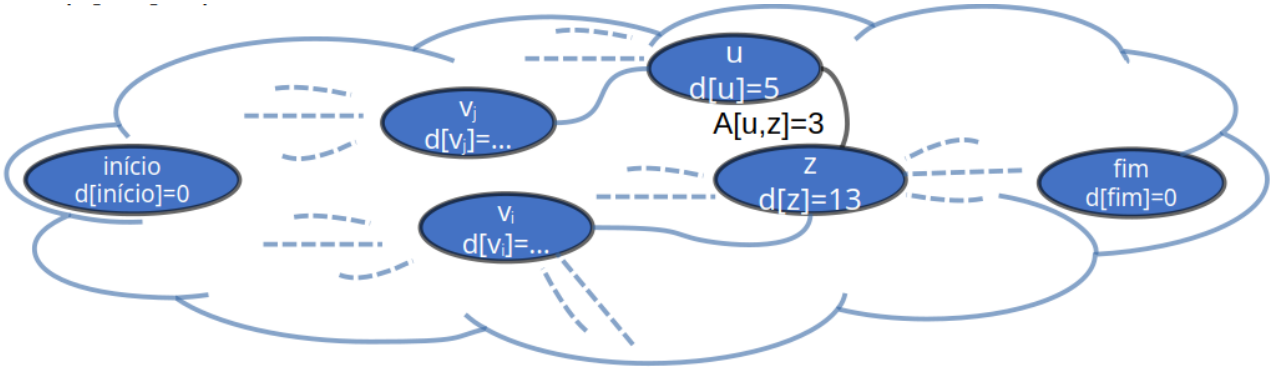
s[z]=u

senão: d[z]=d[z] // não, então mantemos como está

s[z]=s[z]

Ou seja, d[z]=min(d[z],d[u]+A[u,z]) e esse ajuste é conhecido como “relaxação”

Dijkstra



	...	u	z	...
d	...	5	13	...
s	...	v_j	v_i	

No exemplo dado:

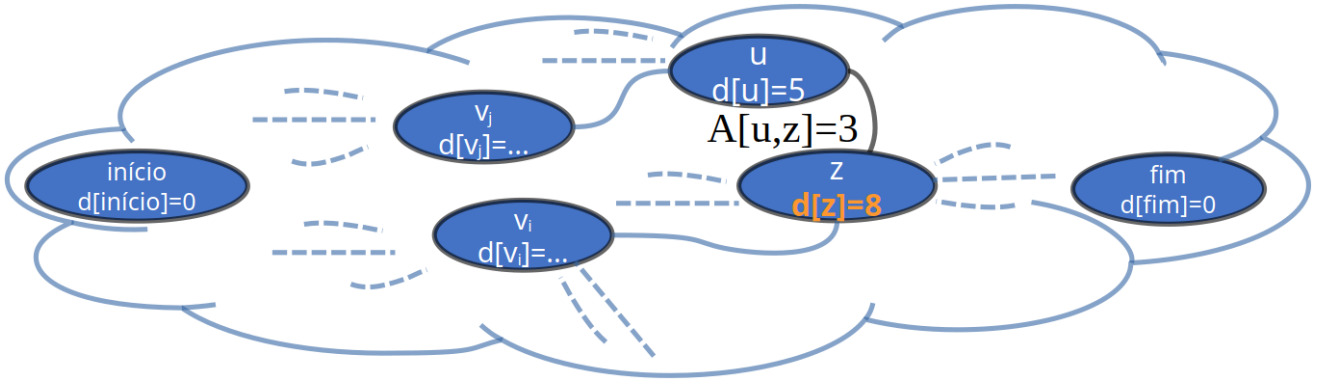
Será que existe um caminho mais curto até *z* via *u* ?

if (13 > 5+3)

então: d[z]=5+3 // sim, então a nova d[z] será aquela obtida via *u*

s[z]= u

Sim, existe um caminho mais curto até *z* via *u.* No nosso exemplo, d[z] será "relaxado" para 8 colocando o vértice u no potencial caminho mínimo passando por z.



	...	u	z	...
d	...	5	8	...
s	...	v_j	u	

Dijkstra

O algoritmo de Dijkstra:

Encontra o caminho mais curto de um nó de origem para um nó de destino em um grafo com pesos de aresta não negativos.

Funciona selecionando iterativamente o nó não visitado com a menor distância atual, adicionando-o a um conjunto "visitado" e, em seguida, "relaxando" (atualizando) as distâncias para seus nós vizinhos se um caminho mais curto for encontrado através do nó atual.

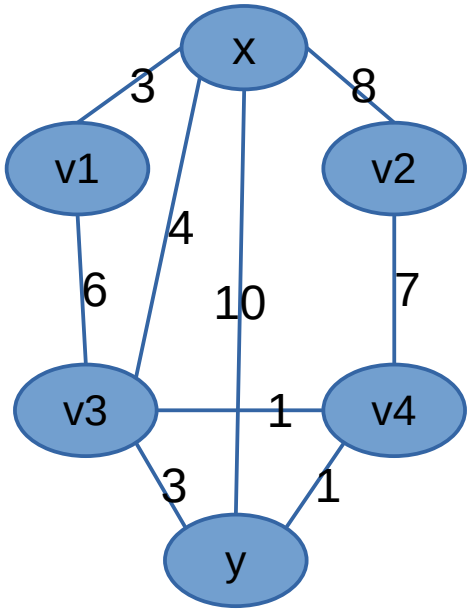
Dijkstra

Dijkstra - Inicialização:

Construa a matriz de adjacências conforme abaixo à esquerda, onde A[i,j] representa o custo/peso da aresta (v_i,v_j) ou ∞ caso não haja tal aresta.

Crie um conjunto de nós visitados (IN) e não visitados (V-IN);

V-IN pode ser implementado como uma fila de prioridade (min-heap)



Matriz de adjacências (A)						
	x	v1	v2	v3	v4	y
x	∞	3	8	4	∞	10
v1	3	∞	∞	6	∞	∞
v2	8	∞	∞	∞	7	∞
v3	4	6	∞	∞	1	3
v4	∞	∞	7	1	∞	1
y	10	∞	∞	3	1	∞

IN V-IN

vértice	x	v1	v2	v3	v4	y
d	0	3	8	4	∞	10
s	–	x	x	x	x	x

d[u] é a distância até o vértice *u.*

s[u] é o vértice que antecede *u* neste caminho.

Dijkstra

ALGORITMO Caminho Mínimo do vértice **x** ao **y** por Dijkstra
Inicializações:
A: matriz de adjacências;
IN: conjunto de vértices; {menor caminho conhecido a partir de x}
d: vetor de inteiros; {distância a partir de x usando os vértices de IN}
s: vetor de vértices; {vértice anterior no caminho mínimo}
begin
(inicia o conjunto IN e os vetores d e s}
IN:= {x}; // vértice de partida
d[x] : = 0; //distância inicial do vértice de partida até ele mesmo

//complementado a inicialização
for todos os vértices z que não pertençam a IN do
    begin
      d[z] := A[x, z]; //distâncias do vértice de partida conforme matriz adj
      s[z] := x; // referencial inicial sempre é o vértice de saída
end;

Dijkstra

Complemento do algoritmo de Dijkstra

```
enquanto y não pertence a IN faça
    p = nó z não pertencente a IN com d[z] mínima
    IN = IN ⋃ {p}
    para todos os nós não pertencentes a IN faça
        DistânciaAnterior = d[z]
        d[z] = min(d[z], d[p] + A[p, z])
        se d[z] < DistânciaAnterior então
            s[z] = p
escreva (“Em ordem inversa, os nós do caminho são”)
escreva (y)
z = y
repita
    escreva (s[z])
    z = s[z]
até z = x
escreva (“A distância percorrida é”, d[y])
```

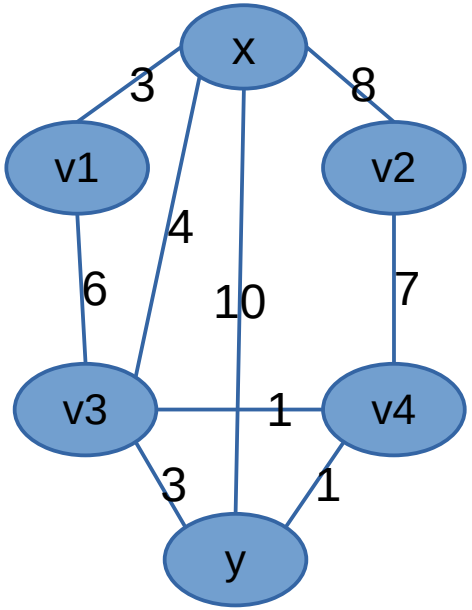
Simulação do algoritmo de Dijkstra (*CaminhoMinimo em* Judith Gersting)

Inicialização*:*

x é o vértice de partida *(s=x)*

Ao final da fase de inicialização teremos IN={x} e *d* contendo todas as distâncias de *x* até os demais vértices em {*V-IN}*.

Entenda "∞" como o maior valor possível para o tipo numérico da distância.



Matriz de adjacências (A)						
	x	v1	v2	v3	v4	y
x	∞	3	8	4	∞	10
v1	3	∞	∞	6	∞	∞
v2	8	∞	∞	∞	7	∞
v3	4	6	∞	∞	1	3
v4	∞	∞	7	1	∞	1
y	10	∞	∞	3	1	∞

IN V-IN

	x	v1	v2	v3	v4	y
d	0	3	8	4	∞	10
s	–	x	x	x	x	x

Tomando a origem em x:

d[u] é a distância até o vértice *u.*

s[u] é o vértice que antecede *u* neste caminho.

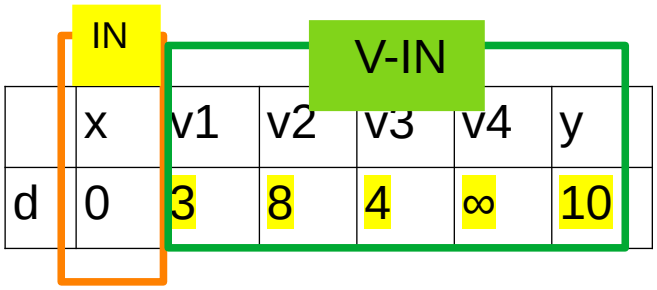
Simulação do algoritmo de Dijkstra (*CaminhoMinimo em* Judith Gersting)

Laço principal (1ª iteração):

ENQUANTO o destino *y* não pertencer a *IN*:

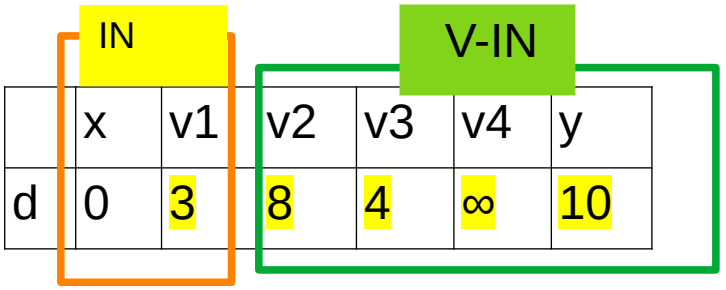
// 1) determine *p*, o vértice com menor valor d[ ] que pertença a V-IN

No caso: p=v1



// 2) atualize *IN = IN U p*

No caso: IN={x,v1} → V-IN={v2,v3,v4,y}



// 3) Agora verifique se há um caminho mínimo chegando em *z em V-IN*

PARA todo *z* em *V-IN*

DistânciaAnterior = d[z]

calcule a relaxação: d[z] = min(d[z],d[p]+A[p,z])

SE d[z] < DistânciaAnterior:

ENTÃO: s[z]=p

Simulação do algoritmo de Dijkstra (*CaminhoMinimo em* Judith Gersting)

Laço principal (1ª iteração):

ENQUANTO o destino *y* não pertencer a *IN*:

No caso: p=v1

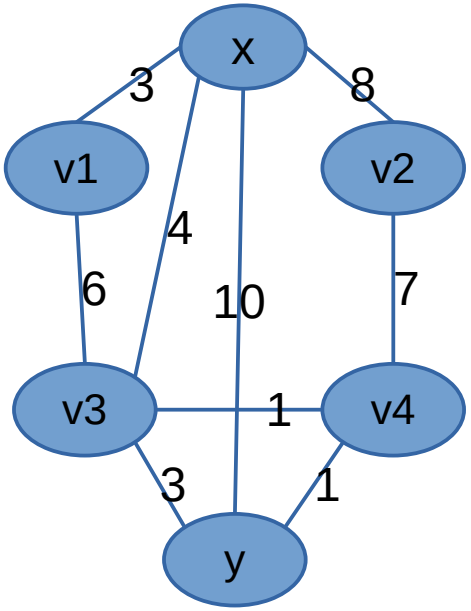
PARA todo *z* em V-IN={v2,v3,v4,y}, d[z] = min(d[z],d[p]+A[p,z])

z=v2: d[v2] = min{d[v2],d[v1]+A[v1,v2]}= min{8,3+∞}=8

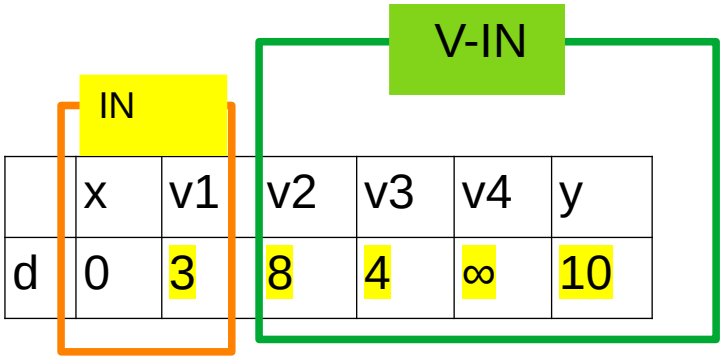
z=v3: d[v3] = min{d[v3],d[v1]+A[v1,v3]}= min{4,3+6}=4

z=v4: d[v4] = min{d[v4],d[v1]+A[v1,v4]}= min{∞,3+∞}=∞

z=y: d[y] = min{d[y],d[v1]+A[v1,y]}= min{10,3+∞}=10



Matriz de adjacências (A)						
	x	v1	v2	v3	v4	y
x	∞	3	8	4	∞	10
v1	3	∞	∞	6	∞	∞
v2	8	∞	∞	∞	7	∞
v3	4	6	∞	∞	1	3
v4	∞	∞	7	1	∞	1
y	10	∞	∞	3	1	∞



	x	v1	v2	v3	v4	y
d	0	3	8	4	∞	10

Simulação do algoritmo de Dijkstra (*CaminhoMinimo em* Judith Gersting)

Laço principal (1ª iteração):

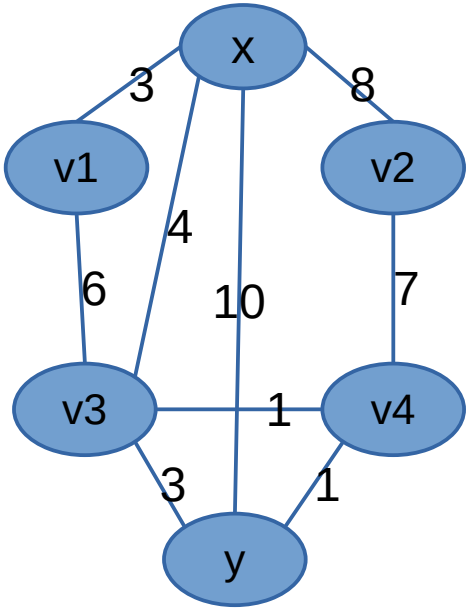
No caso: p=v1 e *z* em V-IN={v2,v3,v4,y}, d[z] = min(d[z],d[p]+A[p,z])

d[v2] = 8
→ manteve a mesma *DistânciaAnterior* → não altere a tabela (d,s)

d[v3] = 4
→ manteve a mesma *DistânciaAnterior* → não altere a tabela (d,s)

d[v4] = ∞
→ manteve a mesma *DistânciaAnterior* → não altere a tabela (d,s)

d[y] = 10
→ manteve a mesma *DistânciaAnterior* → não altere a tabela (d,s)



Os valores na tabela se mantêm inalterados:

IN | V-IN

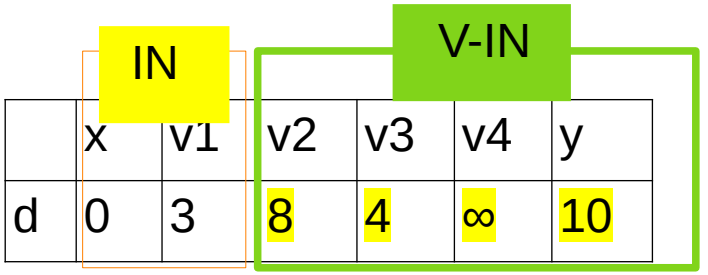
	x	v1	v2	v3	v4	y
d	0	3	8	4	∞	10
s	–	x	x	x	x	x

Simulação do algoritmo de Dijkstra (*CaminhoMinimo em* Judith Gersting)

Laço principal (2ª iteração):

// 1) determine *p*, o vértice com menor valor d[ ] que pertença a V-IN

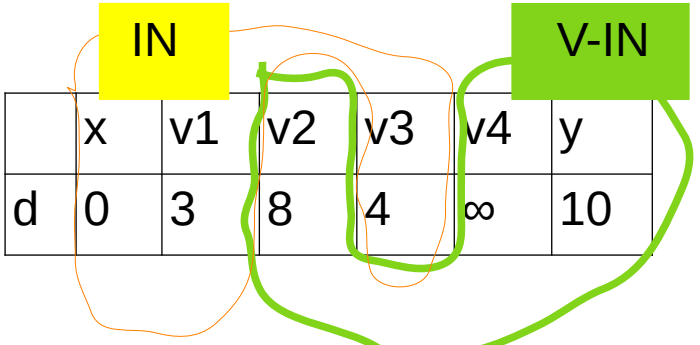
No caso: p=v3



	x	v1	v2	v3	v4	y
d	0	3	8	4	∞	10

// 2) atualize *IN = IN U p*

No caso: IN={x,v1,v3} → V-IN={v2,v4,y}



	x	v1	v2	v3	v4	y
d	0	3	8	4	∞	10

// 3) verifique se há um caminho mínimo chegando em *z em V-IN*

PARA todo *z* em *V-IN*

DistânciaAnterior = d[z]

calcule a relaxação: d[z] = min(d[z],d[p]+A[p,z])

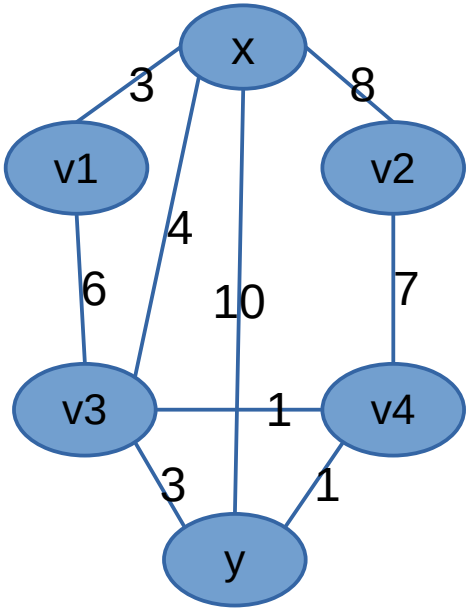
SE d[z] < DistânciaAnterior:

ENTÃO: s[z]=p

Simulação do algoritmo de Dijkstra (*CaminhoMinimo em* Judith Gersting)

Laço principal (2ª iteração):

ENQUANTO o destino *y* não pertencer a *IN*:
No caso: p=v3
PARA todo *z* em V-IN={v2,v4,y}, d[z] = min(d[z],d[p]+A[p,z])
z=v2: d[v2] = min{d[v2],d[v3]+A[v3,v2]}= min{8,4+∞}=8
z=v4: d[v4] = min{d[v4],d[v3]+A[v3,v4]}= min{∞,4+1}=5
z=y: d[y] = min{d[y],d[v3]+A[v3,y]}= min{10,4+3}=7



Matriz de adjacências (A)						
	x	v1	v2	v3	v4	y
x	∞	3	8	4	∞	10
v1	3	∞	∞	6	∞	∞
v2	8	∞	∞	∞	7	∞
v3	4	6	∞	∞	1	3
v4	∞	∞	7	1	∞	1
y	10	∞	∞	3	1	∞

IN

	x	v1	v2	v3	v4	y
d	0	3	8	4	∞	10
s	–	x	x	x	x	x

V-IN

Simulação do algoritmo de Dijkstra (*CaminhoMinimo em* Judith Gersting)

Laço principal (2ª iteração):

No caso: p=v3 e *z* em V-IN={v2,v4,y}, d[z] = min(d[z],d[p]+A[p,z])

d[v2] = 8

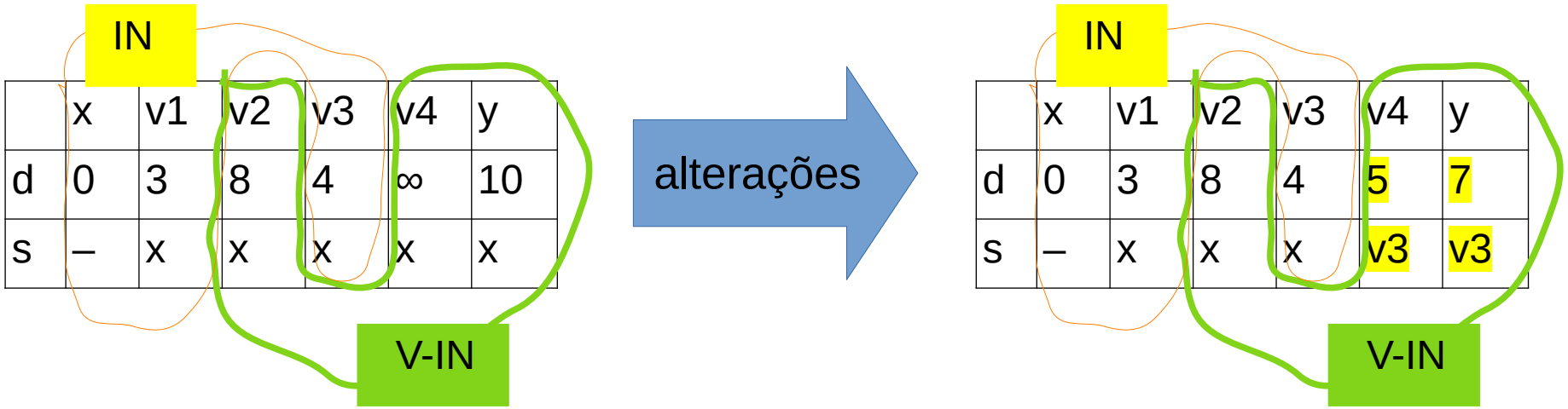
→ manteve a mesma *DistânciaAnterior* → não altere a tabela (d,s)

d[v4] = 5

→ menor do que *DistânciaAnterior* = ∞ → altera a tabela (d,s)

d[y] = 7

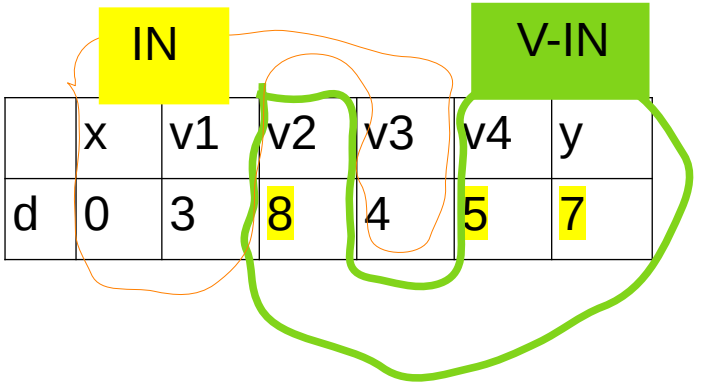
→ menor do que a *DistânciaAnterior* = *10* → altere a tabela (d,s)



Simulação do algoritmo de Dijkstra (*CaminhoMinimo em* Judith Gersting)

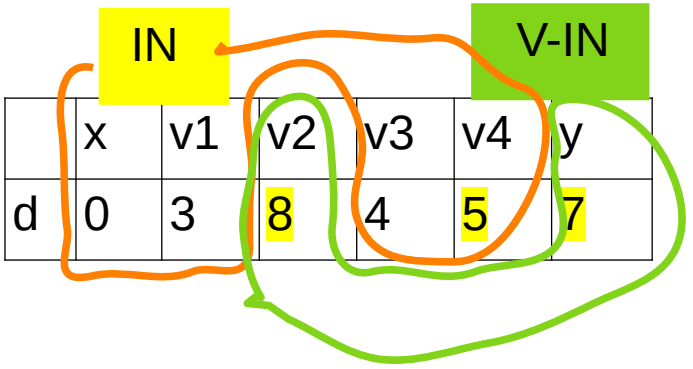
Laço principal (3ª iteração):

// 1) determine *p*, o vértice com menor valor d[ ] que pertença a V-IN
No caso: p=v4



	x	v1	v2	v3	v4	y
d	0	3	8	4	5	7

// 2) atualize *IN = IN U p*
No caso: IN={x,v1,v3,v4} → V-IN={v2,y}



	x	v1	v2	v3	v4	y
d	0	3	8	4	5	7

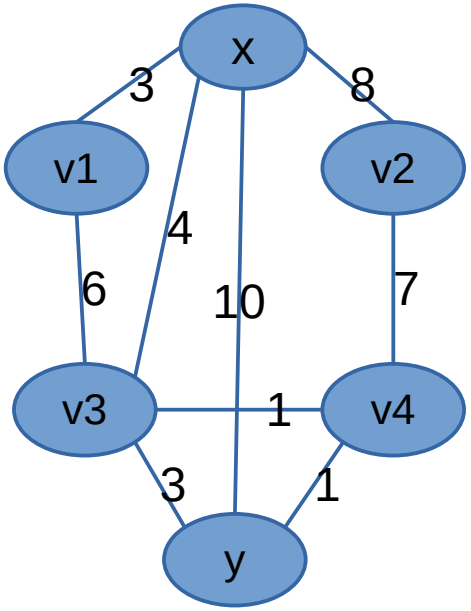
// 3) verifique se há um caminho mínimo chegando em *z em V-IN*
PARA todo *z* em *V-IN*

DistânciaAnterior = d[z]
calcule a relaxação: d[z] = min(d[z],d[p]+A[p,z])
SE d[z] < DistânciaAnterior:
ENTÃO: s[z]=p

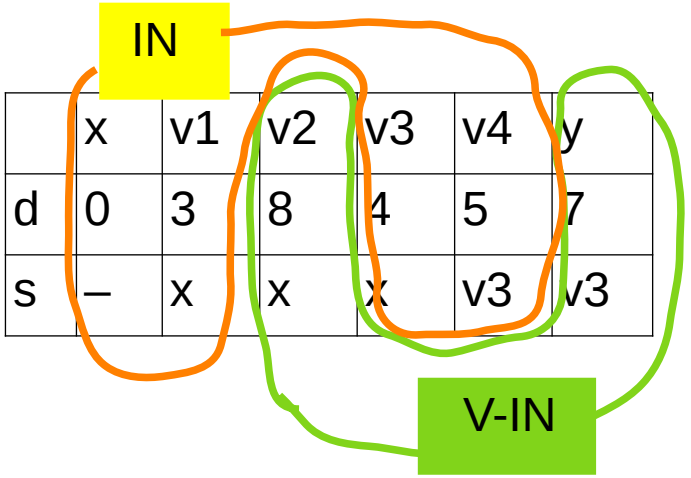
Simulação do algoritmo de Dijkstra (*CaminhoMinimo em* Judith Gersting)

Laço principal (3ª iteração):

ENQUANTO o destino *y* não pertencer a *IN*:
No caso: p=v4
PARA todo *z* em V-IN={v4,y}, d[z] = min(d[z],d[p]+A[p,z])
z=v2: d[v2] = min{d[v2],d[v4]+A[v4,v2]}= min{8,5+7}=8
z=y: d[y] = min{d[y],d[v4]+A[v4,y]}= min{10,5+1}=6



Matriz de adjacências (A)						
	x	v1	v2	v3	v4	y
x	∞	3	8	4	∞	10
v1	3	∞	∞	6	∞	∞
v2	8	∞	∞	∞	7	∞
v3	4	6	∞	∞	1	3
v4	∞	∞	7	1	∞	1
y	10	∞	∞	3	1	∞



	x	v1	v2	v3	v4	y
d	0	3	8	4	5	7
s	–	x	x	x	v3	v3

Simulação do algoritmo de Dijkstra (*CaminhoMinimo em* Judith Gersting)

Laço principal (3ª iteração):

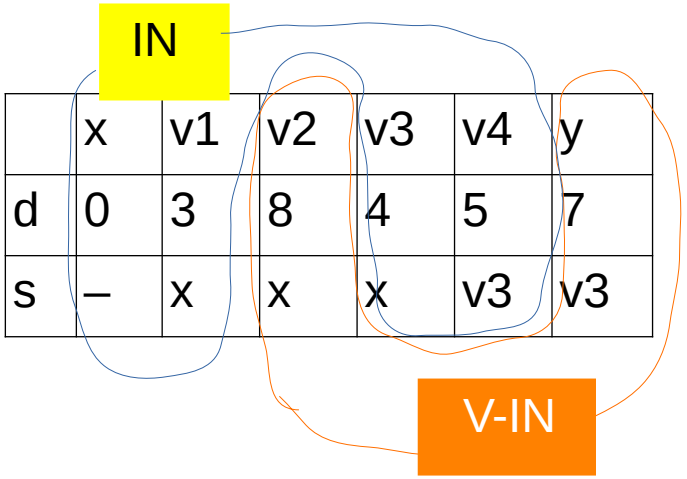
No caso: p=v4 e *z* em V-IN={v2,y}, d[z] = min(d[z],d[p]+A[p,z])

d[v2] = 8

→ manteve a *DistânciaAnterior* → não altera a tabela (d,s)

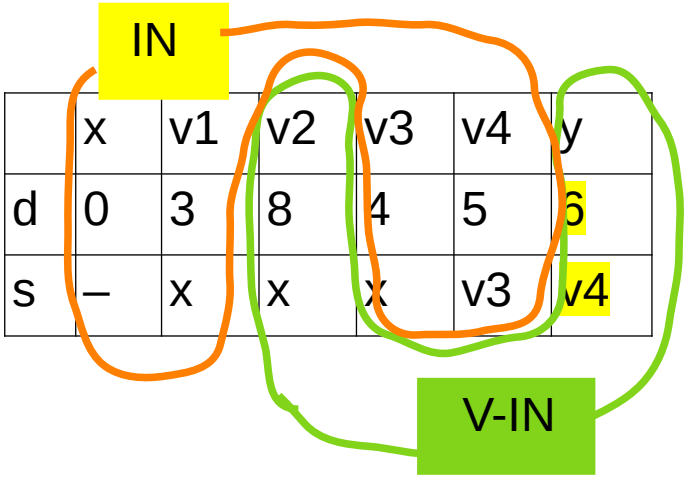
d[y] = 6

→ menor do que *DistânciaAnterior* = *7*→ altere a tabela (d,s)



	x	v1	v2	v3	v4	y
d	0	3	8	4	5	7
s	–	x	x	x	v3	v3





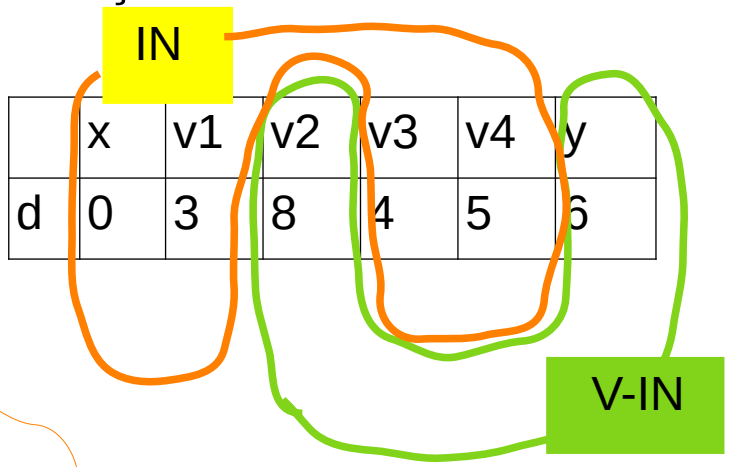
	x	v1	v2	v3	v4	y
d	0	3	8	4	5	6
s	–	x	x	x	v3	v4

Simulação do algoritmo de Dijkstra (*CaminhoMinimo em* Judith Gersting)

Laço principal (4ª iteração):

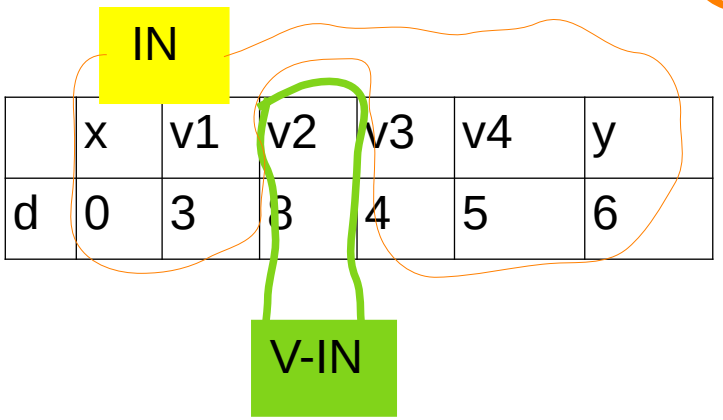
// 1) determine *p*, o vértice com menor valor d[ ] que pertença a V-IN

No caso: p=y



// 2) atualize *IN = IN U p*

No caso: IN={x,v1,v4,v3,y} → V-IN={v2}



// 3) verifique se há um caminho mínimo chegando em *z em V-IN*

PARA todo *z* em *V-IN*

DistânciaAnterior = d[z]

calcule a relaxação: d[z] = min(d[z],d[p]+A[p,z])

SE d[z] < DistânciaAnterior:

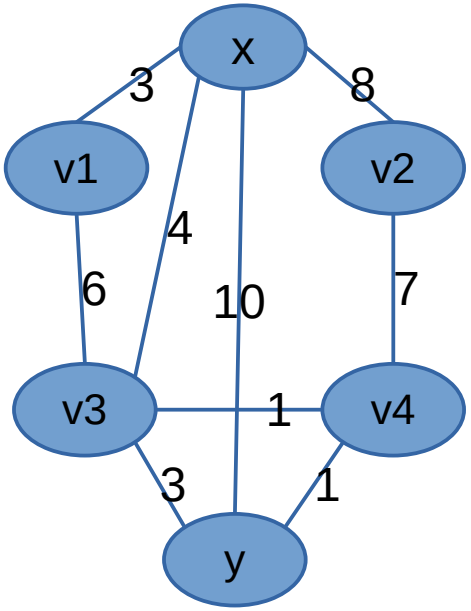
ENTÃO: s[z]=p

Simulação do algoritmo de Dijkstra (*CaminhoMinimo em* Judith Gersting)

Laço principal (5ª iteração):

Não ocorre pois y ∈ IN

ENQUANTO o destino *y* não pertencer a *IN*:
é FALSA a condição que garantiria a iteração



IN

	x	v1	v2	v3	v4	y
d	0	3	8	4	5	6
s	–	x	x	x	v3	v4

V-IN

Agora que y pertence a IN, o laço while termina.

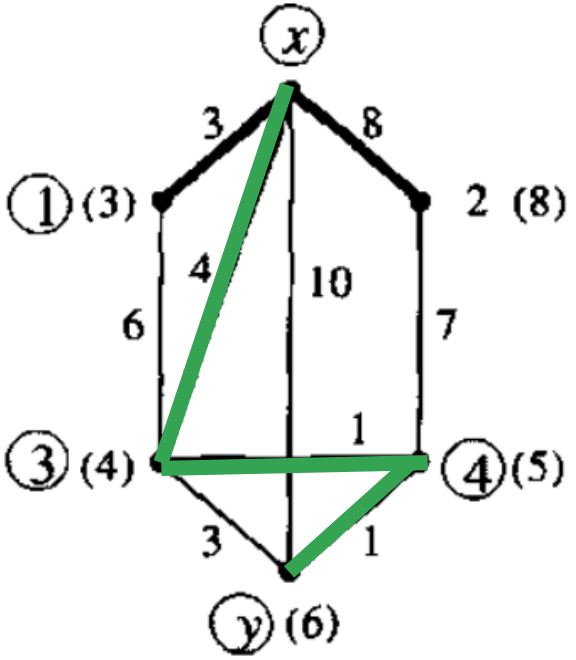
O caminho passa por y, s[y] = 4, s[4] = 3 e .v[3] = x.

Assim, o caminho usa os vértices x, 3, 4 e y.

Note que o algoritmo nos fornece estes vértices na ordem inversa usando a tabela abaixo:

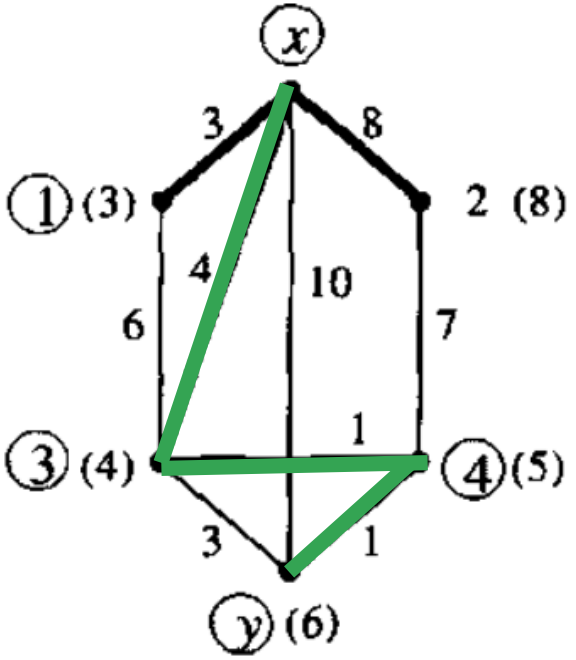
No caso a distância do caminho de custo mínimo é d[y]

	x	1	2	3	4	y
d	0	3	8	4	5	6
s	-	x	x	x	3	4



Experimente esse mesmo grafo no app online https://algo-dijkstra.vercel.app/

	x	1	2	3	4	y
d	0	3	8	4	5	6
s	-	x	x	x	3	4



Caminho mínimo

O Dijkistra é um algoritmo guloso. Ele é classificado dessa forma porque faz a escolha mais otimizada em cada etapa individual com a intenção de encontrar o ótimo global.

Utilizando matriz de adjacências

n = |V|, a complexidade do Dijkstra $O(n^2)$

Podendo ser reduzida para O((V+E)*logV) usando lista de adjacências e uma fila de prioridade min-heap-priority para armazenar vértices ainda não visitados ordenados pelos pesos das arestas.

Tempo para visitar cada vértice = O(V+E)

Tempo para processar cada vizinho de um vértice = O(logV).

total=O(V+E)∗O(logV)=O((V+E)logV)

https://www.scaler.com/topics/data-structures/dijkstra-algorithm/

[CORMEN, 2002]

Caminho mínimo

Conforme já fora mencionado, o algoritmo de Bellman-Ford também resolve o problema de caminho mínimo

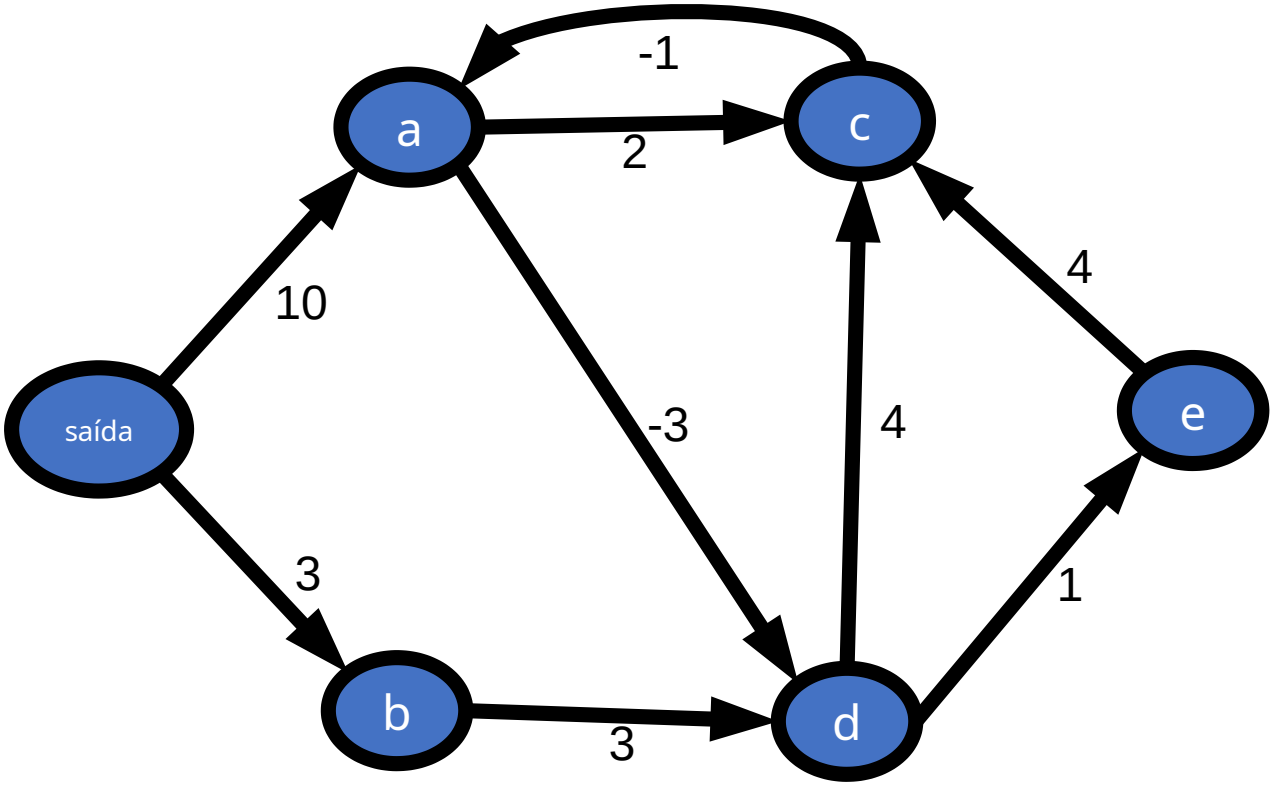
Algumas diferenças entre Bellman-Ford e algoritmo de Dijkstra:

1. Dijkstra trata apenas arestas com pesos positivos. Calculando o caminho de uma fonte a um destino;
2. O algoritmo de Bellman–Ford consegue tratar arestas com pesos negativos.
3. Bellman–Ford indica se existe ou não um ciclo de peso negativo que pode ser alcançado da fonte. Se tal ciclo existe, o algoritmo sinaliza a ausência de solução. Se tal ciclo não existe, o algoritmo produz os caminhos mínimos e seus pesos de uma fonte para todos os outros vértices;

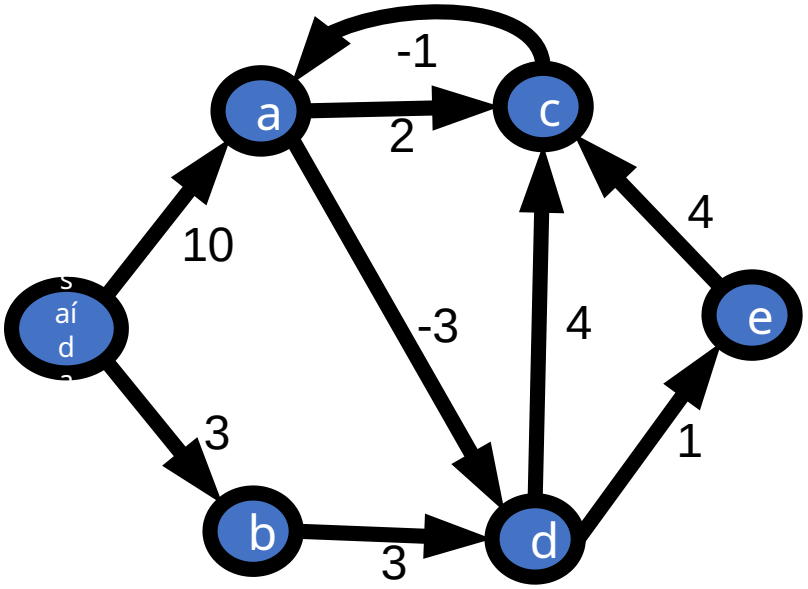
   Se num grafo existe um ciclo cuja soma dos custos das suas arestas é negativa, repetindo este ciclo, pode reduzir-se o comprimento de um caminho tanto quanto se queira [Cardoso et al.].

4. Uma implementação eficiente da estrutura V-IN na forma de uma fila de prioridade min-heap, usada para encontrar o nó com a menor distância em cada etapa, faz o tempo de execução do algoritmo de Dijkstra ser inferior ao do algoritmo de Bellman-Ford [Cormen,2002].

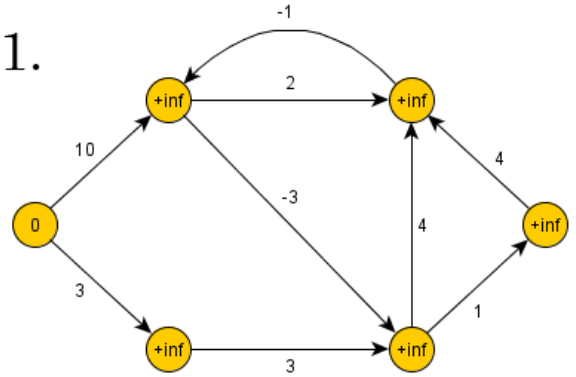
Estude o algoritmo Bellman-Ford (sugestão: pg. 538 do livro da Judith Gersting) e o aplique para encontrar o caminho mínimo entre o vértice “saída” no grafo abaixo:



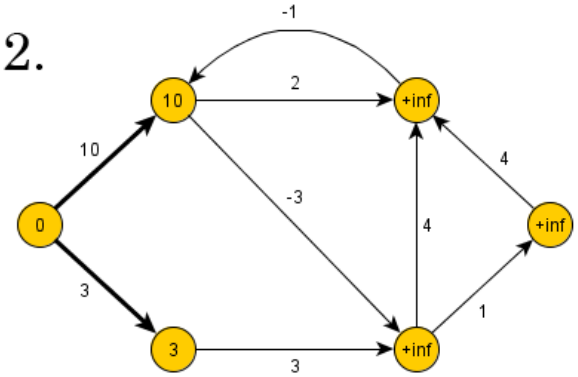
http://www.programming-algorithms.net/article/47389/Bellman-Ford-algorithm



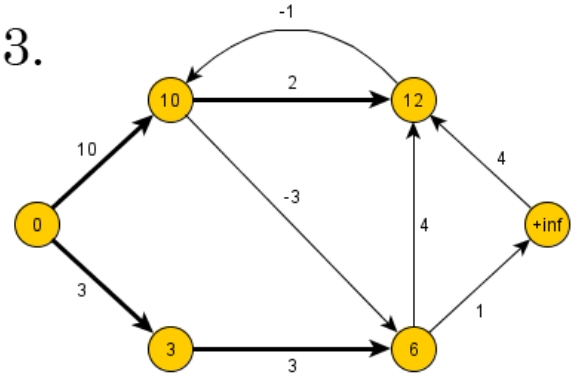
1.



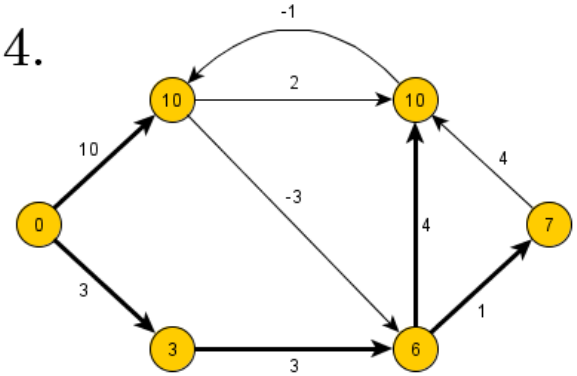
2.



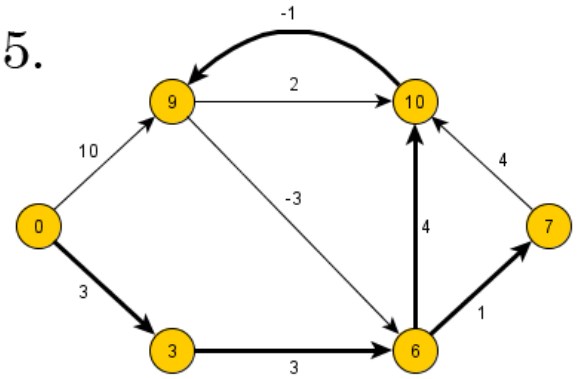
3.



4.



5.



http://www.programming-algorithms.net/article/47389/Bellman-Ford-algorithm

Tem também o Floyd-Warshall

Dijkstra:

- É melhor para problemas de caminho mais curto de origem única em grafos com pesos de aresta não negativos.
- Lida com pesos negativos: Não.
- Detecta ciclos negativos: Não.
- Complexidade de tempo: Geralmente mais rápido, por exemplo, O(E+V log V) com uma fila de prioridade.

Bellman-Ford:

- Melhor para problemas de caminho mais curto de origem única em grafos que podem ter pesos de aresta negativos.
- Lida com pesos negativos: Sim.
- Detecta ciclos negativos: Sim.
- Complexidade de tempo: Mais lento que o Dijkstra, com complexidade O(VE).

Floyd-Warshall:

- Melhor para problemas de caminho mais curto entre todos os pares, onde você precisa do caminho mais curto entre todos os pares possíveis de vértices.
- Lida com pesos negativos: Sim.
- Detecta ciclos negativos: Sim.
- Complexidade de tempo: Mais lento para uma única origem em comparação com o Dijkstra, com complexidade $O(V^3)$. É eficiente para grafos menores ou quando são necessárias informações de todos os pares.

TEG

Bibliografia

Básica

LUCCHESI, C. L. et alli. Aspectos Teóricos da Computação, Parte C: Teoria dos Grafos, projeto Euclides, 1979.

SANTOS, J. P. O. et alli. Introdução à Análise Combinatória. UNICAMP; 1995.

SZWARCFITER, J. L. Grafos e Algoritmos Computacionais. Campus, 1986.

GERSTING, Judith L. Fundamentos Matemáticos para a Ciência da Computação. Rio de Janeiro. 3a Ed. Editora.

Complementar:

1.) CORMEN, T. Introduction to Algorithms, third edition, MIT press, 2009

2.) ROSEN, K. Discrete Mathematics and its applications, seventh edition, McGraw Hill, 2011.

3.) WEST, Douglas, B. Introduction to Graph Theory, second edition, Pearson, 2001.

4.) BONDY, J.A., MURTY, U.S.R., Graph Theory with applications , Springer, 1984.

5.) SEDGEWICK, R. Algorithms in C - part 5 - Graph Algorithms, third edition, 2002, Addison-Wesley.

6.) GOLDBARG, M., GOLDBARG E., Grafos: Conceitos, algoritmos e aplicações. Editora Elsevier, 2012.

7.) BONDY, J.A., MURTY, U.S.R., Graph Theory with applications , Springer, 1984

8.) FEOFILOFF, P., KOHAYAKAWA, Y., WAKABAYASHI, Y., uma introdução sucinta à teoria dos grafos. 2011. (www.ime.usp.br/~pf/teoriadosgrafos)

9.) DIESTEL, R. Graph Theory, second edition, springer, 2000

10.) FURTADO, A. L. Teoria de grafos. Rio de janeiro. Editora LTC. 1973.

11.) WILSON, R.J. Introduction to Graph Theory. John Wiley & Sons Inc., 1985

12.) BOAVENTURA NETTO , P. O. Grafos: Teoria, Modelos, Algoritmos. Edgard Blucher, SP, quinta edição

Tutoriais, artigos, notas de aula...

Vários livros podem ser acessados np formato eletrônico (e-book) via

https://www.udesc.br/bu/acervos/ebook

Exemplos:



Teoria Computacional de Grafos - Os Algoritmos

Jayme Luiz Szwarcfiter



Fundamentos Matemáticos para a Ciência da Computação

Judith L. Gersting



Grafos

Marco Goldbarg



Algoritmos - Teoria e Prática

Thomas Cormen